

Teste da geração de planos e comparação com os planos esperados

Foram testados os cenários da **Situação 1 até a Situação 3**, conforme solicitado. O programa foi executado no SWI-Prolog por meio do comando:

None

```
?- executar_trabalho.
```

A geração dos planos foi feita a partir do estado inicial do mundo dos blocos, considerando os blocos a, b, c e d, com limite horizontal de $X = 0$ até $X = 6$ a fim de garantir coerência física (gravidade + suporte), controle das ações, redução de estados inválidos e execução consistente dos planos.

Regras do Mundo dos Blocos

1. Definição do Espaço

O mundo dos blocos é modelado em um plano bidimensional definido por:

- Um eixo horizontal **X**, limitado ao intervalo discreto de **0 a 6**
- Um eixo vertical **H**, que representa a altura, sem limite superior

O eixo X é dividido em **intervalos unitários**, representando posições possíveis de ocupação:

- (0–1), (1–2), (2–3), (3–4), (4–5), (5–6)

Esses intervalos podem ser ocupados por blocos de diferentes tamanhos, desde que respeitem os limites e não haja sobreposição inválida.

2. Características dos Blocos

O mundo contém quatro blocos, com tamanhos fixos:

- Bloco **a**: tamanho 1
- Bloco **b**: tamanho 1
- Bloco **c**: tamanho 2
- Bloco **d**: tamanho 3

Cada bloco é representado pela estrutura:

bloco(Nome, X_i , X_f , H)

Onde:

- X_i = posição inicial no eixo X
- X_f = posição final no eixo X
- H = altura

3. Estado Inicial

O estado inicial do sistema é definido como:

estado_inicial([

bloco(a, 3, 4, 0),

bloco(b, 5, 6, 0),

bloco(c, 0, 2, 0),

bloco(d, 3, 6, 1)

]).

4. Regras de Apoio e Estabilidade

- A altura $H = 0$ representa o solo, que não requer suporte.
- Para qualquer bloco em $H > 0$, é obrigatório existir uma base completa abaixo dele.
- Um bloco só pode ser posicionado em uma altura superior se **toda a sua extensão em X estiver apoiada**.

Caso contrário, o estado é considerado inválido.

5. Regra de Gravidade

O sistema adota uma regra de gravidade:

Se um bloco não possuir suporte abaixo de sua base, ele automaticamente irá cair para a menor altura possível.

Exemplo:

- Um bloco em (3–6, $H=1$), ao perder o suporte inferior, passa automaticamente para (3–6, $H=0$)

Essa regra é independente do objetivo final e sempre prevalece.

6. Espaços Livres

Um espaço é considerado **livre** quando:

- Não há nenhum bloco ocupando aquela posição em determinada altura
- Existe suporte suficiente (caso $H > 0$)

Observações importantes:

- Espaços no solo ($H = 0$) são sempre válidos se não estiverem ocupados
- Espaços acima só são utilizáveis se houver base completa abaixo
- Um espaço pode estar parcialmente disponível dependendo do tamanho do bloco

7. Movimentação dos Blocos

Existem dois tipos de movimento:

7.1 Movimento Horizontal

- O bloco se desloca ao longo do eixo X
- Só pode ocorrer se:
 - Não houver bloqueio lateral (outros blocos no caminho)
 - O bloco não estiver impedido estruturalmente

Além disso:

Se um bloco possui outros blocos sobre ele, o movimento provoca deslocamento conjunto (efeito de empilhamento).

7.2 Movimento Vertical

- O bloco pode ser reposicionado em qualquer altura válida
- Condições:
 - Não pode haver blocos acima dele
 - O destino deve ser estruturalmente válido (com suporte)

8. Restrições de Movimentação

- Um bloco não pode atravessar outro bloco durante movimentação horizontal
- Um bloco não pode ser movido se houver outro bloco acima dele
- Um bloco só pode ocupar posições livres e válidas

9. Definição de Metas

As metas são definidas como um estado final desejado:

```
estado_final([  
    bloco(d, 3, 6, 0),  
    bloco(a, 4, 5, 1),  
    bloco(b, 5, 6, 1),  
    bloco(c, 4, 6, 2)  
]).
```

10. Estratégia de Prioridade das Metas

Para a resolução dos objetivos, é adotada uma estratégia de prioridade baseada em:

10.1 Nível (Altura)

- Blocos mais próximos do solo (**menor H**) possuem maior prioridade

10.2 Posição Horizontal

- Em caso de empate, priorizam-se blocos mais próximos das extremidades:
 - $X \rightarrow 0$ (esquerda)
 - $X \rightarrow 6$ (direita)

10.3 Tamanho

- Blocos menores possuem maior prioridade

10.4 Critério de Desempate Final

Caso ainda haja empate:

$a > b > c > d$

11. Dependência entre Blocos

A movimentação de blocos deve respeitar dependências estruturais:

- Se um bloco impede o movimento de outro, ele deve ser movido primeiro
- O sistema resolve essas dependências implicitamente ao priorizar ações

Exemplo:

- Para mover um bloco maior para a base, blocos menores que bloqueiam seu caminho devem ser reposicionados antes

12. Consistência do Sistema

Um estado é considerado válido quando:

- Não há sobreposição inválida
- Todos os blocos estão apoiados corretamente
- Nenhum bloco viola as regras de movimentação

Situação 1

Situação 1 — Sf1

Na imagem da Situação 1, o estado final Sf1 mostra o bloco d na base, ocupando a posição de $X = 3$ até $X = 6$, os blocos a e b sobre ele, e o bloco c no topo, ocupando a posição de $X = 4$ até $X = 6$.

```

--- Resolvendo Meta: [bloco(d,3,6,0)] ---
Plano de Execução (Forward): [move(d,0,3,1),move(a,2,3,0),move(d,1,4,1),move(b,0,1,1),move(d,3,6,0)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco d deve estar em (X:3, H:0)
  -> Submeta: Para consolidar o estado, o bloco b deve estar em (X:0, H:1)
  -> Submeta: Para consolidar o estado, o bloco d deve estar em (X:1, H:1)
  -> Submeta: Para consolidar o estado, o bloco a deve estar em (X:2, H:0)
  -> Submeta: Para consolidar o estado, o bloco d deve estar em (X:0, H:1)

--- Resolvendo Meta: [bloco(b,5,6,1)] ---
Plano de Execução (Forward): [move(b,5,6,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco b deve estar em (X:5, H:1)

--- Resolvendo Meta: [bloco(a,4,5,1)] ---
Plano de Execução (Forward): [move(a,4,5,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco a deve estar em (X:4, H:1)

--- Resolvendo Meta: [bloco(c,4,6,2)] ---
Plano de Execução (Forward): [move(c,4,6,2)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco c deve estar em (X:4, H:2)
true
1
0.416 seconds cpu time
```

Meta testada

None

```
[
    bloco(d,3,6,0),
    bloco(b,5,6,1),
    bloco(a,4,5,1),
    bloco(c,4,6,2)
]
```

Plano gerado pelo programa

None

```
[
    move(d,0,3,1),
    move(a,2,3,0),
    move(d,1,4,1),
    move(b,0,1,1),
```

```
move(d,3,6,0),  
  
move(b,5,6,1),  
  
move(a,4,5,1),  
  
move(c,4,6,2)  
  
]
```

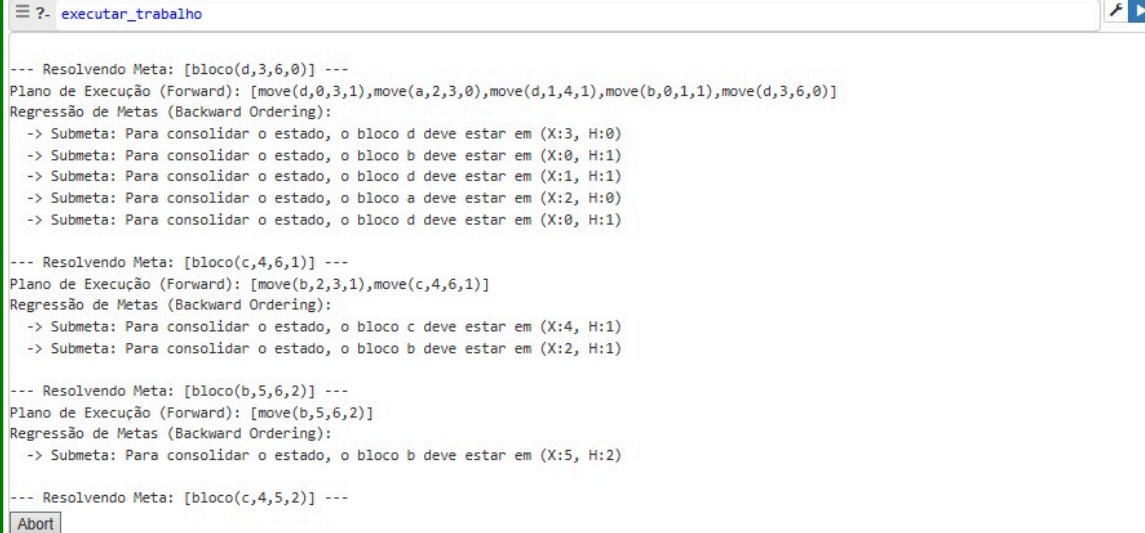
Comparação com o plano manual

O plano gerado atinge o mesmo estado final representado na imagem de Sf1. No plano manual, a ideia principal era liberar a posição da base para que o bloco d ficasse em $X = 3$ até $X = 6$, na altura $H = 0$. Depois disso, os blocos menores seriam colocados sobre ele, formando a pilha final.

O programa segue a mesma finalidade, porém realiza movimentos intermediários para reorganizar os blocos antes de colocar d na base. Assim, o resultado final é compatível com o plano esperado, pois termina com d embaixo, a e b sobre d, e c no topo.

Situação 1 — Sf2

Na imagem da Situação 1, o estado final Sf2 mostra o bloco d na base em $X = 3$ até $X = 6$, o bloco c sobre ele em $X = 4$ até $X = 6$, e os blocos a e b no topo, respectivamente em $X = 4$ até $X = 5$ e $X = 5$ até $X = 6$.



```
??- executar_trabalho  
  
--- Resolvendo Meta: [bloco(d,3,6,0)] ---  
Plano de Execução (Forward): [move(d,0,3,1),move(a,2,3,0),move(d,1,4,1),move(b,0,1,1),move(d,3,6,0)]  
Regressão de Metas (Backward Ordering):  
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:3, H:0)  
-> Submeta: Para consolidar o estado, o bloco b deve estar em (X:0, H:1)  
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:1, H:1)  
-> Submeta: Para consolidar o estado, o bloco a deve estar em (X:2, H:0)  
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:0, H:1)  
  
--- Resolvendo Meta: [bloco(c,4,6,1)] ---  
Plano de Execução (Forward): [move(b,2,3,1),move(c,4,6,1)]  
Regressão de Metas (Backward Ordering):  
-> Submeta: Para consolidar o estado, o bloco c deve estar em (X:4, H:1)  
-> Submeta: Para consolidar o estado, o bloco b deve estar em (X:2, H:1)  
  
--- Resolvendo Meta: [bloco(b,5,6,2)] ---  
Plano de Execução (Forward): [move(b,5,6,2)]  
Regressão de Metas (Backward Ordering):  
-> Submeta: Para consolidar o estado, o bloco b deve estar em (X:5, H:2)  
  
--- Resolvendo Meta: [bloco(c,4,5,2)] ---  
Abort
```

Meta correta de acordo com a imagem

None

```
[  
  bloco(d,3,6,0),  
  bloco(c,4,6,1),  
  bloco(a,4,5,2),  
  bloco(b,5,6,2)  
]
```

Plano gerado pelo programa

Pela imagem do teste, o programa gerou:

None

```
[  
  move(d,0,3,1),  
  move(b,2,3,1),  
  move(c,4,6,1),  
  move(b,5,6,2)  
]
```

Depois, o programa ficou parado ao tentar resolver:

None

```
bloco(a,4,5,2)
```

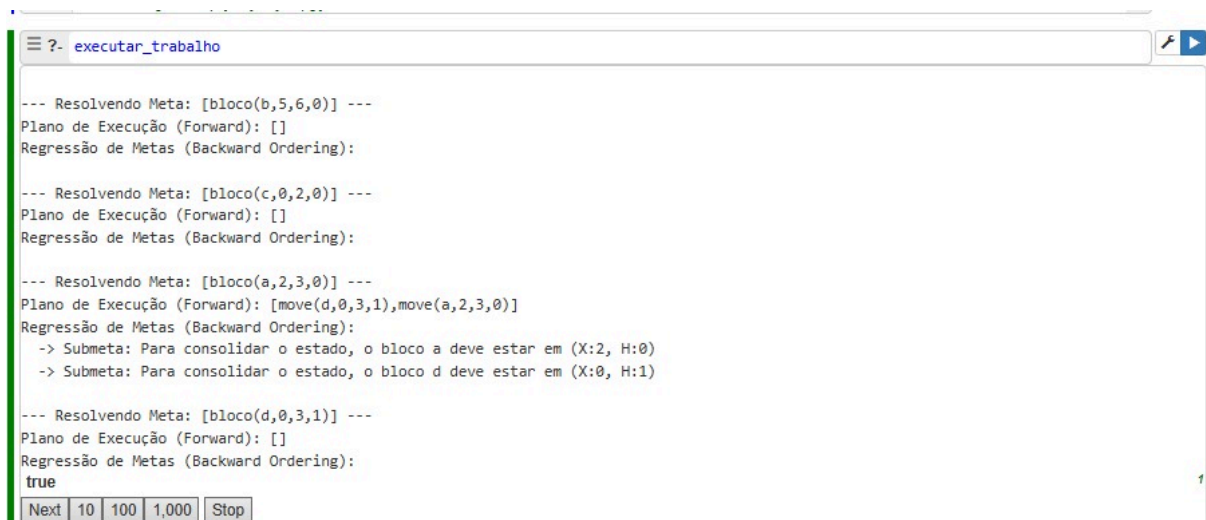
Comparação com o plano manual

O plano manual para Sf2 exige formar uma pilha na direita, com d na base, c sobre d, e a e b no topo. O programa conseguiu avançar até colocar c em $X = 4$ até $X = 6$, na altura $H = 1$, e também conseguiu colocar b em $X = 5$ até $X = 6$, na altura $H = 2$.

Entretanto, o teste não foi finalizado para o bloco a. Portanto, para Sf2, o resultado deve ser registrado como **parcialmente gerado**. Ele segue a mesma direção do plano esperado, mas não conclui totalmente a configuração final mostrada na imagem.

Situação 1 — Sf3

Na imagem da Situação 1, o estado final Sf3 mostra o bloco c na base à esquerda, o bloco a ao lado dele no solo, o bloco d sobre c e a, e o bloco b isolado à direita.



```
--- Resolvendo Meta: [bloco(b,5,6,0)] ---
Plano de Execução (Forward): []
Regressão de Metas (Backward Ordering):

--- Resolvendo Meta: [bloco(c,0,2,0)] ---
Plano de Execução (Forward): []
Regressão de Metas (Backward Ordering):

--- Resolvendo Meta: [bloco(a,2,3,0)] ---
Plano de Execução (Forward): [move(d,0,3,1),move(a,2,3,0)]
Regressão de Metas (Backward Ordering):
-> Submeta: Para consolidar o estado, o bloco a deve estar em (X:2, H:0)
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:0, H:1)

--- Resolvendo Meta: [bloco(d,0,3,1)] ---
Plano de Execução (Forward): []
Regressão de Metas (Backward Ordering):
true
```

Meta testada

None

```
[
    bloco(b,5,6,0),
    bloco(c,0,2,0),
    bloco(a,2,3,0),
    bloco(d,0,3,1)
]
```

Plano gerado pelo programa

None

```
[
```

```
move(d,0,3,1),  
  
move(a,2,3,0)  
  
]
```

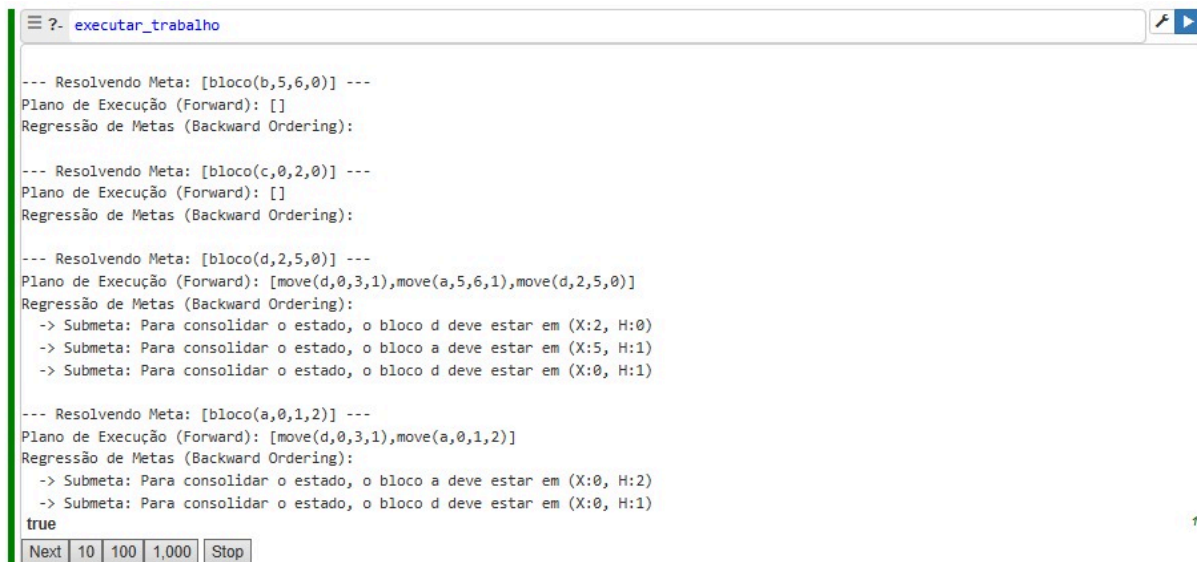
Comparação com o plano manual

O plano gerado corresponde ao estado final Sf3 mostrado na imagem. Os blocos b e c não precisaram ser movidos, pois já estavam em suas posições finais: b em $X = 5$ até $X = 6$, no solo, e c em $X = 0$ até $X = 2$, também no solo.

O programa apenas moveu o bloco d para cima da região esquerda e posicionou o bloco a em $X = 2$ até $X = 3$, completando a base necessária para apoiar d. Assim, o plano gerado coincide com o plano manual para Sf3.

Situação 1 — Sf4

Na imagem da Situação 1, o estado final Sf4 mostra o bloco c na base esquerda, o bloco d no solo ao centro, o bloco b à direita e o bloco a sobre a extremidade esquerda.



```
--- Resolvendo Meta: [bloco(b,5,6,0)] ---  
Plano de Execução (Forward): []  
Regressão de Metas (Backward Ordering):  
  
--- Resolvendo Meta: [bloco(c,0,2,0)] ---  
Plano de Execução (Forward): []  
Regressão de Metas (Backward Ordering):  
  
--- Resolvendo Meta: [bloco(d,2,5,0)] ---  
Plano de Execução (Forward): [move(d,0,3,1),move(a,5,6,1),move(d,2,5,0)]  
Regressão de Metas (Backward Ordering):  
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:2, H:0)  
-> Submeta: Para consolidar o estado, o bloco a deve estar em (X:5, H:1)  
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:0, H:1)  
  
--- Resolvendo Meta: [bloco(a,0,1,2)] ---  
Plano de Execução (Forward): [move(d,0,3,1),move(a,0,1,2)]  
Regressão de Metas (Backward Ordering):  
-> Submeta: Para consolidar o estado, o bloco a deve estar em (X:0, H:2)  
-> Submeta: Para consolidar o estado, o bloco d deve estar em (X:0, H:1)  
true  
Next 10 100 1,000 Stop
```

Meta testada

None

```
[  
    bloco(b, 5, 6, 0),  
    bloco(c, 0, 2, 0),  
    bloco(d, 2, 5, 0),  
    bloco(a, 0, 1, 2)  
]
```

Plano gerado pelo programa

None

```
[  
    move(d, 0, 3, 1),  
    move(a, 5, 6, 1),  
    move(d, 2, 5, 0),  
    move(d, 0, 3, 1),  
    move(a, 0, 1, 2)  
]
```

Comparação com o plano manual

O estado final gerado é compatível com a imagem de Sf4, pois mantém c na base esquerda, posiciona d no centro, mantém b à direita e coloca a sobre a estrutura da esquerda.

No plano manual, a intenção era reorganizar os blocos para liberar o centro e permitir que d ocupasse $X = 2$ até $X = 5$, no solo. O programa também faz isso, mas usa um movimento intermediário adicional do bloco d para conseguir posicionar a na altura final. Portanto, o plano gerado atinge a mesma configuração final do plano manual, embora com uma sequência diferente.

Situação 2

Situação 2 — Estado final S5

```
?- executar_trabalho

--- Resolvendo Meta: [bloco(b,2,3,0)] ---
Plano de Execução (Forward): [move(b,2,3,0)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco b deve estar em (X:2, H:0)

--- Resolvendo Meta: [bloco(a,2,3,1)] ---
Plano de Execução (Forward): [move(a,2,3,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco a deve estar em (X:2, H:1)

--- Resolvendo Meta: [bloco(c,4,6,1)] ---
Plano de Execução (Forward): [move(c,4,6,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco c deve estar em (X:4, H:1)

--- Resolvendo Meta: [bloco(a,4,5,2)] ---
Plano de Execução (Forward): [move(a,4,5,2)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco a deve estar em (X:4, H:2)

--- Resolvendo Meta: [bloco(b,5,6,2)] ---
Plano de Execução (Forward): [move(b,5,6,2)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco b deve estar em (X:5, H:2)

true
```

Meta testada

None

```
[
    bloco(b,2,3,0),
    bloco(a,2,3,1),
    bloco(c,4,6,1),
    bloco(a,4,5,2),
    bloco(b,5,6,2)
]
```

Plano gerado pelo programa

None

```
[
```

```
move(b, 2, 3, 0),  
move(a, 2, 3, 1),  
move(c, 4, 6, 1),  
move(a, 4, 5, 2),  
move(b, 5, 6, 2)  
]
```

Comparação com o plano manual

O plano gerado segue a mesma sequência lógica do plano manual. Primeiro, o bloco b é colocado no solo em $X = 2$ até $X = 3$. Depois, o bloco a é colocado sobre b. Em seguida, o bloco c é posicionado à direita, em $X = 4$ até $X = 6$, na altura $H = 1$. Por fim, os blocos a e b são colocados sobre c, formando a configuração final.

Assim, para a Situação 2, o plano automático corresponde ao plano manual.

Situação 3

Situação 3 — Estado final S7

```

? executar_trabalho

--- Resolvendo Meta: [bloco(d,0,3,1)] ---
Plano de Execução (Forward): [move(d,0,3,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco d deve estar em (X:0, H:1)

--- Resolvendo Meta: [bloco(a,5,6,1)] ---
Plano de Execução (Forward): [move(a,5,6,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco a deve estar em (X:5, H:1)

--- Resolvendo Meta: [bloco(d,2,5,0)] ---
Plano de Execução (Forward): [move(d,2,5,0)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco d deve estar em (X:2, H:0)

--- Resolvendo Meta: [bloco(a,0,1,1)] ---
Plano de Execução (Forward): [move(a,0,1,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco a deve estar em (X:0, H:1)

--- Resolvendo Meta: [bloco(b,1,2,1)] ---
Plano de Execução (Forward): [move(b,1,2,1)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco b deve estar em (X:1, H:1)

--- Resolvendo Meta: [bloco(b,1,2,1)] ---
Plano de Execução (Forward): []
Regressão de Metas (Backward Ordering):

--- Resolvendo Meta: [bloco(d,3,6,0)] ---
Plano de Execução (Forward): [move(d,3,6,0)]
Regressão de Metas (Backward Ordering):
  -> Submeta: Para consolidar o estado, o bloco d deve estar em (X:3, H:0)
true
Next 10 100 1,000 Stop
```

Meta testada

None

```
[
    bloco(d,0,3,1),
    bloco(a,5,6,1),
    bloco(d,2,5,0),
    bloco(a,0,1,1),
    bloco(b,1,2,1),
    bloco(d,3,6,0)
]
```

Plano gerado pelo programa

None

```
[  
  move(d,0,3,1),  
  move(a,5,6,1),  
  move(d,2,5,0),  
  move(a,0,1,1),  
  move(b,1,2,1),  
  move(d,3,6,0)  
]
```

Comparação com o plano manual

O plano gerado acompanha a mesma sequência geral do plano manual. Primeiro, o bloco d é deslocado para a esquerda. Depois, o bloco a é reposicionado na direita. Em seguida, o bloco d volta para o solo em uma posição intermediária. Após isso, os blocos a e b são colocados sobre a região esquerda. Por fim, o bloco d é movido para sua posição final em $X = 3$ até $X = 6$, na altura $H = 0$.

No teste exibido, apareceu uma meta repetida para o bloco b:

None

```
bloco(b,1,2,1)
```

Por esse motivo, uma das etapas retornou plano vazio [], pois a meta já estava satisfeita. Desconsiderando essa repetição, o plano gerado corresponde ao plano manual para a Situação 3.

Síntese da comparação

Cenário	Resultado do teste	Comparação com o plano manual
---------	--------------------	-------------------------------

Situação 1 — Sf1	Plano gerado com sucesso	Alcança a mesma configuração da imagem: d na base, a e b sobre d, e c no topo
Situação 1 — Sf2	Plano gerado parcialmente	Segue a direção do plano manual, mas não conclui a posição final de a no topo
Situação 1 — Sf3	Plano gerado com sucesso	Coincide com o plano manual, pois apenas d e a precisaram ser movidos
Situação 1 — Sf4	Plano gerado com sucesso	Alcança a configuração final da imagem, apesar de usar movimentos intermediários diferentes
Situação 2 — S5	Plano gerado com sucesso	Corresponde ao plano manual
Situação 3 — S7	Plano gerado com sucesso	Corresponde ao plano manual, desconsiderando a meta repetida de b

Resposta final

A geração automática de planos foi testada para todos os cenários até a Situação 3. Na Situação 1, os estados finais Sf1, Sf3 e Sf4 foram resolvidos com sucesso e chegaram às mesmas configurações indicadas nas imagens. O caso Sf2 foi gerado apenas parcialmente, pois o programa avançou na formação da pilha da direita, mas não concluiu o posicionamento final do bloco a.

Na Situação 2, o plano gerado correspondeu ao plano manual até o estado S5, seguindo a ordem esperada de movimentação dos blocos. Na Situação 3, o programa também gerou um plano compatível com o plano manual até S7, havendo apenas uma repetição da meta do bloco b, o que produziu uma etapa com plano vazio.

Portanto, os testes mostram que os planos gerados pelo programa são compatíveis com os planos manuais na maior parte dos cenários, alcançando as configurações finais representadas nas imagens corretas das situações.

